Nova Bank Branch Automation System
UML Design

User

User Interacting with View

<<package>> View

BankApplication

accountController: AccountController

+ showMainMenu() : void
+ openAccount() : void
+ depositMoney() : void
+ withdrawMoney() : void
+ closeAccount() : void
+ calculateInterest() : void
+ showDailyTransactions() : void

Uses

<<package>> Controller

AccountController

accountService: AccountService

+ openAccount(account: Account) : boolean
+ deposit(accountNumber: String, amount: double) : boolean
+ withdraw(accountNumber: String, amount: double) : boolean
+ closeAccount(accountNumber: String) : boolean
+ calculateInterest(accountNumber: String) : double
+ showDailyTransactions(date: LocalDate) : Transaction[ ]

Uses

<<package>> Service

AccountService

accountDAO: AccountDAO
transactionDAO: TransactionDAO

+ openAccount(account: Account) : boolean
+ depositMoney(accountNumber: String, amount: double) : boolean
+ withdrawMoney(accountNumber: String, amount: double) : boolean
+ closeAccount(accountNumber: String) : boolean
+ calculateInterest(accountNumber: String) : double
+ generateDailyTransactionReport(date: LocalDate) : Transaction[ ]

Uses

<<package>> DAO (Data Access Object)

DAO when we manage **collections of objects**.

<<interface>> TransactionDAO

+ addTransaction(transaction: Transaction) : boolean
+ getTransactionsByAccount(accountNumber: String) : Transaction[ ]
+ getTransactionsByDate(date: LocalDate) : Transaction[ ]
+ getAllTransactions() : Transaction[ ]

Implements

TransactionDAOImpl

accountDAO : AccountDAO

+ addTransaction(transaction: Transaction) : boolean
+ getTransactionsByAccount(accountNumber: String) : Transaction[ ]
+ getTransactionsByDate(date: LocalDate) : Transaction[ ]
+ getAllTransactions() : Transaction[ ]

Uses

<<interface>> AccountDAO

+ addAccount(account: Account) : boolean
+ getAccountByNumber(accountNumber: String) : Account
+ updateAccount(account: Account) : boolean
+ deleteAccount(accountNumber: String) : boolean
+ getAllAccounts() : Account[ ]

Implements

AccountDAOImpl

branch : BankBranch

+ addAccount(account: Account) : boolean
+ getAccountByNumber(accountNumber: String) : Account
+ updateAccount(account: Account) : boolean
+ deleteAccount(accountNumber: String) : boolean
+ getAllAccounts() : Account[ ]

Manages

<<package>> Model

BankBranch

branchId: int;
branchName: string;
branchAddress: String;
accountArray: Account[ ];
accountCount: int;

+ default constructor
+ parametarized constructor
+ Getters / Setters
+ toString

1 — has-a — *

(Abstract) Account

accountNumber: String;
accountHolderName: String;
accountOpeningDate: LocalDate;
accountClosingDate: LocalDate;
interestRate: double;
currentBalance: double;
accountStatus: AccountStatus;
transactionArray: Transaction[ ];
transactionCount: int;

+ default constructor
+ parametarized constructor
+ Getters / Setters
+ toString
+ abstract deposit(amount: double): boolean
+ abstract withdraw(amount: double): boolean
+ abstract calculateInterest(): double
+ addTransaction(t: Transaction): void {final}

<<enumeration>> AccountStatus
ACTIVE
FROZEN
CLOSED

1 — has-a — *

Transaction

transactionId: String;
transactionType: TransactionType;
amount: double;
transactionDate: LocalDateTime;

+ default constructor
+ parametarized constructor
+ Getters
+ toString

<<enumeration>> TransactionType
DEPOSIT
WITHDRAW
INTEREST
LOAN_REPAYMENT

Extends

SavingAccount

static final MINIMUM_BALANCE: double;

+ default constructor
+ parametarized constructor
+ Getters
+ toString
+ deposit(amount : double) : boolean
+ withdraw(amount : double) : boolean
+ calculateInterest() : double

Extends

CurrentAccount

maxOverdraftLimit: double

+ default constructor
+ parametarized constructor
+ Getters / Setters
+ toString
+ deposit(amount : double) : boolean
+ withdraw(amount : double) : boolean
+ calculateInterest() : double

Extends

LoanAccount

final loanAmount: double
loanTermMonths: int
installmentAmount: double
interestType: InterestType

<<enumeration>> InterestType
FIXED
FLOATING
REDUCING

+ default constructor
+ parametarized constructor
+ Getters
+ toString
+ deposit(amount : double) : boolean
+ withdraw(amount : double) : boolean {Operation not supported}
+ calculateInterest() : double

method is implemented to return false or throw an exception.

Extends

SalaryAccount

lastTransactionDate: LocalDate;

+ default constructor
+ parametarized constructor
+ Getters
+ toString
+ deposit(amount : double) : boolean
+ withdraw(amount : double) : boolean
- checkAccountInactivity(): void